Group: Randy Amparo, Amerie Rodriguez, and Pascal Aiddou

CIS 344

11/4/2026

Insurance Claim Processing System Documentation

Project Plan (Randy Amparo):

The purpose of this project was to create a secure system that employees of an insurance firm can use to access, process, and store customer data like claims, policies, and approvals. Firstly, when our group started brainstorming ideas about the design, we agreed on creating a system similar to an intranet. This system would store data and perform procedures that employees of an insurance firm would need. One key feature that we first focused on was an employee login page. To log in, employees would have to use login information that is stored in the insurance_firm_344.sql database. For example, employee Randy Amparo would log in with his email address and password to access the claim processing system. If the email address and password that he enters on the login page don't match the email_address and e_password columns saved under his row, it would deny him access. Our system was created in a way that requires emails and passwords assigned to employees to create security. Another feature that we planned on creating was claim creation. On top of being able to approve, deny, and review claims that are created, employees also have the permission to create new claims on behalf of customers. For example, if there were new customers that needed to be added to the system, any employee could enter their first name, last name, policy type, incident date, claim amount, and description. A highlight feature that made our system more complex was making sure claims that

were reviewed recorded the name of the reviewer. When employees approve or deny claims, the decision is recorded in the status column, as well as the decision dates.

Database Management:

The application that we created uses SQL joins to retrieve data using the pending_claims.php and view_claims.php files. Firstly, the pending_claims.php joins claims to policies and customers using a three-table JOIN for the review page of the system. This shows the customers' names and coverage_type along with each claim under review. Secondly, the view_claims.php adds LEFT JOIN to employees to show who reviewed the claim. The reason for the LEFT JOINwas to keep the claim in the results even if there's no matching reviewer.

PHP Backend Documentation (Amerie Rodriguez):

When making the PHP end of the application, I thought of the outline of the project first while simultaneously looking at the key requirements. I wanted the user to be able to login, log out, add a claim, view claims, and approve or deny pending claims.

First, I started with the login page (*login.php*) . I knew that I wanted users to be able to go through a secure login page, where their user follows them while they're navigating through the session, so I started off with session_start();, which allows the user to navigate through the webpages and the website remembers who the user is. Next I used a bit of code that allows for my PHP files/code to interact with mySQL, so that I may call upon it freely. Then I made a query that uses methods that separates the code from the data the user entered, making SQL injection impossible. I used these methods: $stmt = $db->prepare($query);, $stmt->bind_param();, $stmt->execute();, $stmt->bind_result(); and if

($stmt->fetch()) throughout my coding process so SQL injection is impossible. To avoid redundancy, I won't reiterate the parts containing these methods in later explanations.

Then, I made a page to add new claims (*add_claim.php*), while this part was a little tricky, I managed to get through it by requesting the user to give me the customers full name, what coverage they're receiving, when the incident happened, the claim amount, and the claim description in a form. I did have a bit of trouble with the customer part since I didn't want to make a new table, or make some complex code, I decided it would be better to make defaults. I used a transaction to allow me to enter information from the form into the corresponding tables (c_first_name to Customer first_name). All new claims automatically go to pending claims to be approved or denied.

Then I made pending claims (*pending_claims.php*), my intention was to make it so that when a user submits a claim, it comes to a pending claims webpage where an employee must approve or deny this claim, based on the decision a payment is recorded if it is approved. $query = "UPDATE Claims SET decision_status = 'Denied', decision_date = ?, reviewed_by = ? WHERE claim_id = ?"; will update a claim decision status to be denied while also marking the decision date and who reviewed it. The same goes for the approvals, except there is a $db->begin_transaction(); which makes it so that when an employee approves a claim the payment must also be recorded, BOTH steps (approving & recording payment) must succeed, if anything fails it will $db->rollback(); and erase any changes.

Then I wanted the user to view claims on a join table, so I made (*view_claims.php*), this is basically a joins table, the query selects the claim ID, customer names, policy coverage type, claim amount, description, decision status and date, and reviewing employee names from the Claims base table. It is joined to the Policies table on matching policy IDs, further joined to

the Customers table on matching customer IDs, and left-joined to the Employees table on matching reviewer IDs. These are only showing me approved/denied claims because claims under review go to pending_claims.php. Then I use an if else block and while loop to display the join table, if there are no approved or denied claims then it will echo (or show the user) that there are no processed claims yet.

Lastly, I made the logout page (*logout.php*) which was pretty simple, you just have to do session_destroy(); which destroys the session.

Front-end Documentation (Pascal Aiddou):

For this project, our group was focused on making an insurance claim processing system. Our goal was to create a website that would let insurance company employees access customer information and manage insurance claims (approve or deny).

I was tasked with frontend development, which meant I handled the HTML/CSS. With this in mind, I wanted to create something rather simple while still making it look good. I began by creating the employee login page using HTML. The login page consisted of input fields for an email address and password, whilst also having a button to submit the form. After completing this, I moved on to CSS. CSS would help the page come to life and overall make the project feel like a real website. I started by creating a separate CSS file so I could target specific parts of the page's appearance. When creating the CSS file for the login page, I wanted to go with a green and white theme, so that's exactly what I did.

I started the CSS by focusing on the body. I changed the background color, font, margins, and text alignment. I also decided to center everything for a cleaner look.

Then I looked into the form. I made it width relative to make it not take up too much room and used margins to center it on the page. I added a little more space between the email and password fields to make them look less close to each other. For the input boxes, I added some padding and border radius to make them a little prettier and added dark green borders to correspond with the color scheme I'd chosen. I added a simple hover effect to the button so that when the user puts their mouse over the button, the background would change to a dark green and the text would change to white. I thought this made the page look a little more animated without overdoing it. When I completed the login page, I chose to keep the same green/white style for the rest of the pages on the website. I wanted the login page not to look too separate from the other pages, since they were all on the same system. So I made a main menu CSS file and then a CSS file for the new claims page, the claims history page, and the pending claims page. Even though each page had a different aim, I tried to keep the colours and fonts the same.

As for the main menu, I wanted the options for navigation to look more like buttons and less like a list of links. I used list-style none to eliminate the default circle that appears for list items and then chose a width for the list that would keep everything centered. I also added space between the options to keep them from being located too close to each other. For the links, I chose a white background color, dark green for the border, and rounded corners. I also used the same effect I had used for the login button of switching the color to a dark green in the background and using the color white for the text when hovering over it. I thought that this helped clean up the appearance of the menu, while also helping the user to identify which option they are choosing. Finally, I chose the color green for the logout option so that it would be consistent with the rest of the page.

Next, I worked on the CSS for the new claims page. Since this had more input fields than the login page, I wanted to ensure the form was still clear and easy to fill out. I made the form a little wider and increased the spacing between the different input fields. I designed the dropdown menu and the input boxes so they have the same width and spacing, green borders, and round corners. This ensured that the page still looks professional even with the different types of input boxes. I again styled the submit button to have a white background and a green border, with the same hover effect. Finally, I changed the colour of the link for the employee back to the main menu to match the theme.

The page for the claim's history was a little different because it was mostly a table rather than a form. I wanted the information to be clearly laid out, so I paid particular attention to the width of the table, the borders, and the space around the items. I set the width at 90% and used auto margins to centre the table on the page. The table's layout was further improved by using border-collapse, collapse, which resulted in the borders not appearing doubled between different items. The table heading has a dark green background with white text to make it clear to the user that the information below was a different set of data, and I set the padding in the table cells so the words are separated from the border a little more, and used the same dark green colour for the table border to make the information easier to follow.

I then worked on the pending claims page. This page was very similar to the claims history page, but it also included approve and deny buttons. Since both pages were in a table, I kept the styling of the page close to the same. I used the same width of the table, the same dark green borders and padding, and the same dark green background with white headings. However, I had to style the approve and deny buttons as well. I added a white background, a green border, a border-radius of 7px, and a padding of 2px on the buttons. Likewise with the other buttons

across the website, I added a hover effect to the buttons that changed the background to dark green and the text to white.

I aimed to keep all the CSS files consistent throughout. I kept to the same style of a sans-serif font, white background, and dark green colouring for borders, links, and hover state. I kept the content mostly in the centre, and used a combination of margins and padding to create space between elements. This is despite the pages being entirely different content types; I wanted the user to feel like they were moving through the same whole system.

In general, I wanted to keep the website looking professional, neat, and simple without including too many crazy features. Since employees will primarily be using this application to view information and submit and approve/deny claims, I didn't think the fancy visual details were necessary. All of the different CSS files allowed me to become comfortable styling forms, tables, buttons, and links for a page. Green and white were used throughout the entire website in order to establish a cohesive look, and I think by doing so I was able to keep all of the pages looking consistent while still being tailored towards each individual function.